NCAS Unified Model Introduction: Practical sessions (Rose/Cylc)

NCAS-CMS

February 2026

PRACTICALS

Practical exercises for UM training course.

Can also be used for self-study.

NCAS Computational Modelling Services: http://cms.ncas.ac.uk/

SECTION

ONE

TO DO LIST

Todo

Find another suite for this task

(The original entry is located in /Users/ros/work/git-projects/um-training/practicals/source/rose-cylc-exercises.rst, line 25.)

SECTION

TWO

GETTING SET UP (LEEDS TRAINING COURSE)

Warning

You **MUST** have PUMA2, ARCHER2 and MOSRS accounts setup before starting this section.

Warning

These instructions are for use on the in-person UM Training Course held in Leeds. If you are using them for self-study please contact NCAS-CMS for instructions.

2.1 Set up your ARCHER2 connection - Using your own laptop

If you are using your own laptop, you should already have setup so that you can access ARCHER2 from a teminal window, please proceed to the section *Set up your ARCHER2 environment*.

2.2 Set up your ARCHER2 connection - Using an NCAS laptop

To use the UM Introduction Tutorials you will first need to ensure you can connect from your local desktop to ARCHER2.

2.2.1 SSH key files

Before you try and connect to ARCHER2, you need to make sure that you have the ssh-keys available on your computer. In these instructions, we've assumed the keys are called `id_rsa_archer2` and `id_rsa_archer2.pub`. Replace with the name of your keys as appropriate.

Hopefully you remembered to bring your ARCHER2 ssh-key with you on a USB stick. Please talk to a course tutor if you have forgotten it.

Copy your ssh-key from the USB stick to the `~/.ssh` directory.

2.2.2 Connecting via a Terminal (GNU/Linux & macOS)

Connecting via a terminal with an X11 connection (XQuartz is also required when using macOS)

Login to ARCHER2:

```
ssh -Y -i ~/.ssh/id_rsa_archer2 <archer2-username>@login.archer2.ac.uk
```

To simplify the login process, you can define a `~/.ssh/config` file entry containing the necessary information. For example:

```
Host archer2
Hostname login.archer2.ac.uk
User <archer2_username>
IdentityFile ~/.ssh/id_rsa_archer2
ForwardX11 yes
ForwardX11Trusted yes
```

so that you can connect using just the command:

```
ssh archer2
```

2.3 Set up your ARCHER2 environment

Login to ARCHER2 from your local desktop, copy the following profile to your home directory.

```
archer2$ cp /work/y07/shared/umshared/um-training/rose-profile ~/.profile
```

Change the permissions on your `/home` and `/work` directories to enable the NCAS-CMS team to help with any queries:

```
chmod -R g+rX /home/n02/n02/<your-username>
chmod -R g+rX /work/n02/n02/<your-username>
```

2.4 Set up your PUMA2 connection

PUMA2 is accessed from the ARCHER2 login nodes, and you will use the same username and password.

From an ARCHER2 terminal type:

```
archer2$ ssh -Y puma2
```

and type your ARCHER2 password when prompted.

You should now be logged into PUMA2. To go back to the ARCHER2 login nodes, type `exit`.

2.4.1 Set up passwordless access to PUMA2

You can set up a passphrase-less ssh-key to allow you to connect to PUMA2 without typing a password or passphrase.

> **Note**
>
> We would never normally advise using an ssh-key without a passphrase, but in this case it is safe to do so since we are already authenticated within the ARCHER2 system.

From the ARCHER2 login nodes, type:

```
archer2$ ssh-keygen -t rsa -f ~/.ssh/id_rsa_puma2
```

At the prompt, press enter for an empty passphrase.

Copy the key over to PUMA2:

```
archer2$ ssh-copy-id -i ~/.ssh/id_rsa_puma2 puma2
```

Type in your ARCHER2 password when prompted.

Next, create a file called `~/.ssh/config` (if it doesn't already exist), and add the following lines:

```
Host puma2
IdentityFile ~/.ssh/id_rsa_puma2
ForwardX11 yes
```

Test it works by typing:

```
archer2$ ssh puma2
```

You should not be prompted for your password. Note that this should have set up X11 forwarding, so you no longer need the `-Y` option.

> **Warning**
>
> You should never use a passphrase-less key to access the ARCHER2 login nodes, as this is a serious security risk.

2.5 Set up your PUMA2 environment

Copy our standard `.profile` and `.bashrc` files:

```
puma2$ cd
puma2$ cp ~um1/um-training/puma2/.bash_profile .
puma2$ cp ~um1/um-training/puma2/.bashrc .
```

Logout of PUMA2 and back in again to pick up these changes.

You will get a warning about not being able to find `~/.ssh/ssh-setup`. This can be ignored and will be resolved in the next step.

You should also be prompted for your Met Office Science Repository Service password, then username. Note that it asks for your **password** first.

Remember your MOSRS username is one word; usually firstnamelastname, all in lowercase.

If the password caching works, you should see:

```
Subversion password cached
Rosie password cached
```

This means you can now access the code and roses suites stored in the Met Office respositories.

> **Note**
>
> The cached password is configured to expire after 12 hours. Simply run the command `mosrs-cache-password` to re-cache it if this happens. Also if you know you won't need access to the repositories during a login session then just press return when asked for your MOSRS password.

Finally, change the permission on your PUMA2 `/home` space:

```
chmod -R g+rX /home/n02/n02/<your-username>
```

2.6 Set up your ssh-agent

In order to submit jobs to ARCHER2 from PUMA2, you will need to set up an `ssh-agent` and use it to cache the passphrase to your ARCHER2 key.

i. Copy your ARCHER2 ssh-key pair to PUMA2

Your ARCHER2 key is the one that you use to ssh into the ARCHER2 login nodes. You need to copy both the public and private keys into your `.ssh/` directory on PUMA2.

Open a new terminal from wherever you originally connected to ARCHER2 in *Set up your ARCHER2 connection - Using your own laptop*, and run the following command

```
scp ~/.ssh/id_rsa_archer2* <archer2-username>@login.archer2.ac.uk:/home/n02/n02-
↪puma/<archer2-username>/.ssh
```

ii. Start up your ssh-agent

First copy the `ssh-setup` script to your `.ssh/` directory.

```
puma2$ cp ~um1/um-training/setup/ssh-setup ~/.ssh
```

Next log out of PUMA2 and back in again to start up the `ssh-agent` process. You should see the following message

```
Initialising new SSH agent...
```

iii. Add your ARCHER2 key

Add your ARCHER2 key to the `ssh-agent`, by running

```
puma2$ ssh-add ~/.ssh/id_rsa_archer2
```

Enter your passphrase when prompted. If the passphrase has been cached successfully you should see a message like this:

```
Identity added: /home/n02/n02/<archer2-username>/.ssh/id_rsa_archer2
```

The `ssh-agent` will continue to run even when you log out of PUMA2. However, it may stop from time to time, for example if PUMA2 is rebooted. For instructions on what to do in this situation see *Restarting your ssh agent* in the Appendix.

iv. Configure access to the ARCHER2 login nodes

Create a file `~/.ssh/config` (if it doesn't already exist), and add the following lines:

```
# ARCHER2 login nodes
Host ln*
IdentityFile ~/.ssh/id_rsa_archer2
```

iv. Verify the setup is correct

To test this is all working correctly, run:

```
puma2$ rose host-select archer2
```

This should return one of the login nodes, e.g. `ln01`. If it returns a message like `[WARN] ln03: (ssh failed)` then something has gone wrong with the ssh setup.

2.7 Using the Cylc Web GUI

Cylc provides a terminal based user interface and a web browser based one. The course can be completed using either. If you wish to use the Cylc Web GUI some setup is required. Here we provide instructions for Linux, Mac & MobaXterm (Windows).

2.7.1 Linux/Mac

Edit the `~/.ssh/config` file on your laptop so that the ARCHER2 and PUMA2 sections contain:

```
# PUMA2
Host puma2
User <username>
IdentityFile ~/.ssh/<your-archer-ssh-key>
ProxyJump archer2

# ARCHER2
Host archer2
Hostname login.archer2.ac.uk
User <username>
IdentityFile ~/.ssh/<your-archer-ssh-key>
ForwardX11 No
ControlMaster auto
ControlPath /tmp/ssh-socket-%r@%h-%p
ControlPersist yes
```

You should now be able to type `ssh puma2` and land directly on PUMA2.

Setup an alias on your laptop by adding one for the following to your `~/.bashrc` or equivlaent depending on whether you are using Linux or a Mac.

i. Linux

```
alias puma-ui='PORT=$(shuf -n 1 -i 10000-65000); ssh -t -L ${PORT}:localhost:$
↪{PORT} puma2 "bash -l -c \"export CYLC_VERSION=8; cylc gui --no-browser --
↪Application.log_level=WARN --port-retries=0 --port=${PORT}\""'
```

ii. Mac

```
alias puma-ui='PORT=$(jot -r 1 10000 65000); ssh -t -L ${PORT}:localhost:${PORT}␣
↪puma2 "bash -l -c \"export CYLC_VERSION=8; cylc gui --no-browser --Application.
↪log_level=WARN --port-retries=0 --port=${PORT}\""'
```

2.7.2 MobaXterm (Windows)

i. Launch MobaXterm

ii. Open a terminal (click on the "+" tab)

iii. If needed create a `~/.ssh` directory

iv. Copy your ARCHER2 key into `~/.ssh` on MobaXterm the local disk is available at /mnt/c

v. Edit (or create) the file `~/.ssh/config` with the following contents:

```
# PUMA2
Host puma2
User <username>
IdentityFile ~/.ssh/<your-archer-ssh-key>
ProxyJump archer2

# ARCHER2
Host archer2
Hostname login.archer2.ac.uk
User <username>
IdentityFile ~/.ssh/<your-archer-ssh-key>
ForwardX11 No
ControlMaster auto
ControlPath /tmp/ssh-socket-%r@%h-%p
ControlPersist yes
```

Setup an alias by adding the following line to your `~/.bashrc`:

```
alias puma-ui='PORT=$(shuf -n 1 -i 10000-65000); ssh -t -L ${PORT}:localhost:$
↪{PORT} puma2 "bash -l -c \"export CYLC_VERSION=8; cylc gui --no-browser --
↪Application.log_level=WARN --port-retries=0 --port=${PORT}\""'
```

2.7.3 Start up the cylc web GUI

In a new terminal window type:

```
$ puma-ui
```

You see see a response similar to the following:

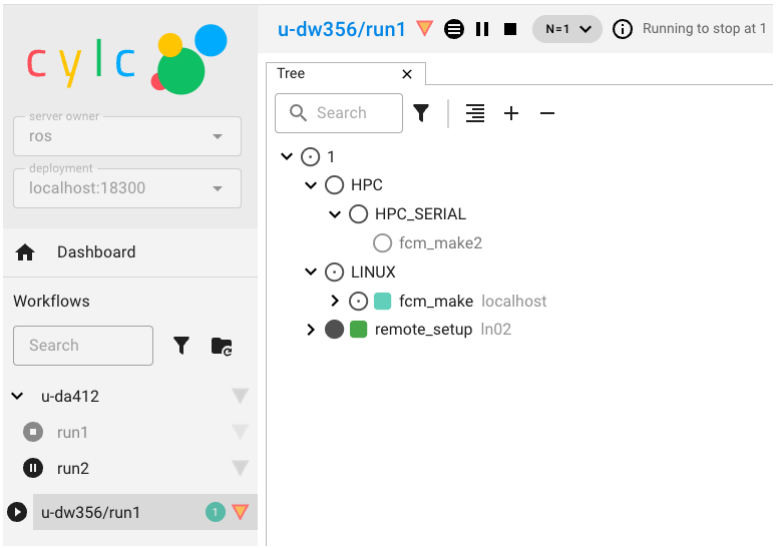
```
$ puma-ui
################################################################################
------------------------------Welcome to PUMA2--------------------------------
################################################################################
[C 2026-01-22 08:25:30.367 ServerApp]

To access the server, open this file in a browser:
```

(continues on next page)

(continued from previous page)

```
  file:///home/n02/n02/ros/.cylc/uiserver/info_files/jpserver-1094362-open.html
Or copy and paste one of these URLs:
  http://localhost:20522/cylc?
↪token=700ab2be96800177d03df31b8140857cab02b9632af45a1d
  http://127.0.0.1:20522/cylc?
↪token=700ab2be96800177d03df31b8140857cab02b9632af45a1d
[W 2026-01-22 08:25:44.242 ServerApp] The websocket_ping_timeout (999999) cannot␣
↪be longer than the websocket_ping_interval (290).
  Setting websocket_ping_timeout=290
```

Copy and paste one of the http URLs listed into your web browser and you should then see your Cylc GUI load. If you have never used Cylc8 before the Workflows panel will be empty.



You are now ready to try running a UM suite!

SECTION

THREE

WORKING WITH SUITES

Aims

In this section you will learn:

- How to find, download and edit Rose suites

3.1 Suite Discovery and Management: rosie go

Important

The `rosie go` GUI is being phased out and will be replaced with command line tools in the next version of the Rose software. `rosie go` will be available on PUMA2 until the end of November 2026.

Rosie go is the suite discovery GUI. From `rosie go` it is possible to search for suites, checkout, copy and delete suites.

Launch `rosie go` by typing:

```
puma2$ rosie go
```

3.1.1 Searching for suites

By default, `rosie go` will show all the suites that you have checked out locally, so unless you have used Rose before you will have an empty results panel to begin with. You can search for suites by typing a word/phrase into the *Search* box and either click the *Search* button or press <Enter>. The search looks for the entered word/phrase in **any** of a suite's properties.

- Enter **u-ag137** in the *Search* box and press <Enter>

You should now see a long list of suites. All these suites are listed as they reference `u-ag137` in their suite information.

More advanced queries can be run by clicking the + button next to the Search button. Queries allow you to filter results based on the values of particular properties. You can combine filters to make complex queries.

- Select **idx** in the property box and **eq** in the operator box and then enter the value **u-ag137**. Click *Query*.

This time you should only see one id listed in the results pane.

- Now run a query to list all suites **owned by rosalynhatcher containing ARCHER2 in their title**.

The searches you run within `rosie go` are recorded in your search history.

- View your history by clicking *History –> Show search history*.

The *Search History* panel should now be displayed on the left-hand side of `rosie go` listing your past searches. You can order the results by type, parameters or whether you asked to see all revisions by clicking on the relevent column head. To re-run one of these searches simply click on it.

- Try running your initial search for `u-ag137` again.

Close the *Search History* Panel by clicking the close button (*X*) in the top-right of the panel.

3.1.2 Viewing suite information

To obtain more information about a suite listed in the search results you can do one of two things:

1. Hold your mouse over the suite id to display a tooltip containing more details
2. Right click on the suite and select *Info* in the pop-up menu to display a dialog box containing further details

Search results can be ordered by property in either ascending or descending order. To do so, click on the column title for the property you wish to order by so an arrow is displayed next to it indicating the order in which the property is being sorted.

3.1.3 Checking out an existing suite

Right-clicking on a suite displays a pop-up menu from which you can perform many functions on the suite; e.g. checkout, copy, delete, etc. We will perform many of these actions in subsequent exercises but to begin with we will just checkout an existing suite to use in the *Editing Suites/Workflows: rose edit* exercises. To view a suite it must be checked out first.

- Right-click on suite `u-ag137` and select *Checkout Suite* from the pop-up menu.

When you checkout a suite it is always placed in your `~/roses` directory. In this state, the suite is simply a working copy - you can edit it and run it but any changes you make will only be held locally.

Note

You can also checkout a suite by highlighting it and then clicking the *Checkout* button on the toolbar or by running `rosie checkout <suiteid>` on the puma2 command-line.

3.1.4 Other useful features

To see what suites you have checked out click the *Show local suites* button to the left of the search box (represented by the *house* icon). You should have at least 1 suite listed.

Question

- What do you think the *house* icon in the local column indicates?

3.2 Editing Suites/Workflows: rose edit

The `rose config editor` in combination with the metadata file, which describes UM inputs, is the GUI for editing UM suites. Building and running the UM under Rose requires, at least, two separate apps: an `fcm_make` app to build the model executable and a `um` app to configure the runtime namelists and environment variables. Coupled models may require additional `fcm_make` apps, one for each executable to be built.

3.2.1 Launch the config editor GUI

Right click on suite `u-ag137` and select *Edit Suite*. The `rose edit` GUI will start up.

On the left hand side is a navigation panel containing a tree listing the apps in the suite configuration. For this particular suite these are:

- *suite conf* - General suite configuration options
- *fcm_make_pp* - Extract and build the post-processing scripts
- *fcm_make_um* - Extract and build the UM source code
- *housekeeping* - Tidies up log files, old work and data directories
- *install_ancil* - Install ancillary files
- *postproc* - Post-processing settings
- *rose_ana* - Rose built in app; used here for comparison of dump files
- *rose_arch* - Rose built in app; used here for archiving of log files
- *um* - The UM atmosphere and reconfiguration settings

3.2.2 Explore the GUI

Click on the triangle to the left of *suite conf* to expand that section. Click on *Build and run switches*. A panel will appear on the right-hand side containing options for controlling what tasks will be run for this workflow. You can see that it will build the UM and reconfiguration executables, run the reconfiguration and then run the model.

> **Note**
>
> We generally use a **common notation** to help users navigate through the GUI and to help us help you with questions. Getting to "UM Science Settings" would be indicated like this: *um –> namelist –> UM Science Settings*. This notation will be used throughout the rest of this tutorial.

The input namelists for the UM are contained in the *um –> namelist* section. Let's take a look at the science namelist for *Microphysics (Large Scale Precipitation)*, `run_precip` under *UM Science Settings*.

For each UM namelist item there is a short description to help you understand what that variable is. Click on the cog next to a namelist variable and select *Help* to view more detailed information. The help information can give you some useful pointers but be aware that it has been written with Met Office setup in mind.

Range and type checking of variables is done as soon as the user enters a new value. Try changing the value of `timestep_mp_in` to `0`. This will cause an error flag to appear, hover over the error for more information and click the *undo* button several times to revert to the original value.

Some larger science sections have been been divided into subsections; take a look at *Section 05 - Convection* for an example of this. To open a section in a new tab click with the middle mouse button, expand the

section by clicking the page triangles. Rose edit has a search box which can be used to search item names. Try searching for the variable `astart` where the input dump is specified, you will be taken directly to the *Dumping and Meaning* panel.

Trigger ignored settings are hidden by default and only appear to the user when the appropriate options are selected. Open the *Gravity Wave Drag* panel, if you change `i_gwd_vn` from `5` to `4` the options available change. Click the *save* button to apply these changes to your app. Let's take a look at what effect this has had to the `rose-app.conf` file, run `fcm diff` in the suite directory.

```
puma2$ cd ~/roses/u-ag137
puma2$ fcm diff -g
```

You should see that several namelist items have had `!!` added to the start of the line. This tells Rose to ignore these items when processing the app file into Fortran namelists. Should you wish to see all variables on a panel select *View All Ignored Variables* and *View Latent Variables* from the *View* menu.

Switch back to the Rose edit window and click the *Undo* button to revert the changes and then *Save* the suite again. To view all changes made to the suite configuration in the current session go to *Edit* > *Undo/Redo Viewer*.

3.2.3 Error checking of UM inputs

In addition to the type and range checking of namelist items and environment variables, more thorough checks can be made using Rose macros and the fail-if/warn-if metadata.

First let's check if the suite contains any options which trigger the fail-if and warn-if checks in the UM metadata. Select menu item *Metadata* > *Check fail-if, warn-if*. As this suite is setup correctly `FailureRuleChecker: No problems found` should appear at the bottom right of the window.

Now let's try and introduce both a warning and a failure. We're going to change the boundary layer option `alpha_cd`. Either navigate to *Section 03 - Boundary Layer –> Implicit solver options* or type `alpha_cd` into the search bar. Click on the `+` sign to add an array element to `alpha_cd` and type `1.5` into the new box. Next navigate to *Reconfiguration and Ancillary Control –> Output dump grid sizes and levels* and increase the number of ozone levels to `86`. Now run the fail-if, warn-if checker again.

Question

- What is the error?
- What is the warning?

Use the *Undo* button to put the settings back to how we found them and run the checker again. It is strongly recommended that whenever namelists and environment variables are modified that the fail-if, warn-if checker is applied before running the workflow.

SECTION

FOUR

RUNNING A UM WORKFLOW ON ARCHER2

Aims

In this section you will learn:

- How to run and monitor a standard UM workflow
- Where to find the job output and error files
- Where to find the UM model output

4.1 ARCHER2 architecture

In common with many HPC systems, ARCHER2 consists of different types of processor nodes:

- **Login nodes:** This is where you land when you ssh into ARCHER2. Typically these processors are used for file management tasks.
- **Compute / batch nodes:** These make up most of the ARCHER2 system, and this is where the model runs.
- **Serial / post-processing nodes:** This is where less intensive tasks such as compilation and archiving take place.

ARCHER2 has three file systems:

- **/home:** This is relatively small and is only backed up for disaster recovery.
- **/work:** This is much larger, but is not backed up. Note that the batch nodes can only see the work file system. It is optimised for parallel I/O and large files.
- **/a2fs-nvme:** This is high performance solid state storage configured as temporary SCRATCH storage where files not accessed in the last 28 days are automatically deleted. It is not backed up.

Further Reading

Consult the ARCHER2 website for more information: http://www.archer2.ac.uk

4.2 Running a Standard Workflow

To demonstrate how to run the UM through Rose we will start by running a standard N48 workflow at UM13.8.

4.2.1 Copy the suite

- In `rosie go` locate the suite with idx **u-dp063** owned by **grenvillelister**.
- Right click on the suite and select *Copy Suite*.

This copies an existing suite to a new suite id. The new suite will be owned by you. During the copy process a wizard will launch for you to edit the suite discovery information, if you wish.

A new suite will be created in the MOSRS rosie-u suite repository and will be checked out into your `~/roses` directory.

4.2.2 Edit the suite configuration

- Open your new suite in the Rose config editor GUI.

Before you can run the workflow you need to change the *userid*, *queue*, *account code* and *reservation*:

- Click on *suite conf –> template variables* in the left hand panel
- Set `HPC_USER` (that's your ARCHER2 username)

If following the tutorial as part of an organised training event:

- Set `HPC_ACCOUNT` to **'n02-training'**
- Set `HPC_QUEUE` to **'standard'**
- Ensure `RESERVATION` is set to **True**
- Set `HPC_RESERVATION` to be the reservation code for today. (e.g. **'n02-training_1673632'**)

If following the tutorial as self-study:

- Set `HPC_ACCOUNT` to the budget code for your project. (e.g. **'n02-cms'**)
- Set `HPC_QUEUE` to **'short'**
- Set `RESERVATION` to **False**

Notes

- Quotes around the variable values are essential otherwise the workflow will not run.
- In normal practice you submit your workflow to the parallel queue (either **short** or **standard**) on ARCHER2.
- For organised training events, we use processor reservations, whereby we have exclusive access to a prearranged amount of ARCHER2 resource. Reservations are specified by adding an additional setting called the reservation code; e.g. **n02-training_226**.

- Save the suite (*File* > *Save* or click the *down arrow* icon)
- Close the *rose edit* window.

4.2.3 Run the workflow

Run the following commands to submit the workflow:

```
puma2$ cd ~/roses/u-dp063
puma2$ cylc vip
```

This validates, installs and runs the workflow. The standard workflow will build, reconfigure and run the UM.

Tip

If no argument is supplied to `cylc vip`, the current working directory is used as the source directory. Alternatively, you can supply a workflow source name e.g. `cylc vip u-dp063`

Further Information

`cylc vip` is short for **validate**, **install** and **play** (run) the workflow.

- Validate checks the suite definition for errors.
- Install creates the *run directory* structure along with some service files.
- Play runs the workflow.

4.3 Monitor the running workflow

To help you monitor and control running workflows Cylc offers 3 tools:

- A web based graphical user interface (Cylc GUI)
- A terminal-based user interface (Cylc TUI)
- A comprehensive command line interface (Cylc CLI)

See also

For full details see the Cylc user interfaces documentation

These pages give information on how to navigate around both GUIs and indicates what all the task & job icons indicate.

4.3.1 Using the Cylc User Interfaces

We will take some time here to explore the Cylc User Interfaces

Using the Cylc web GUI

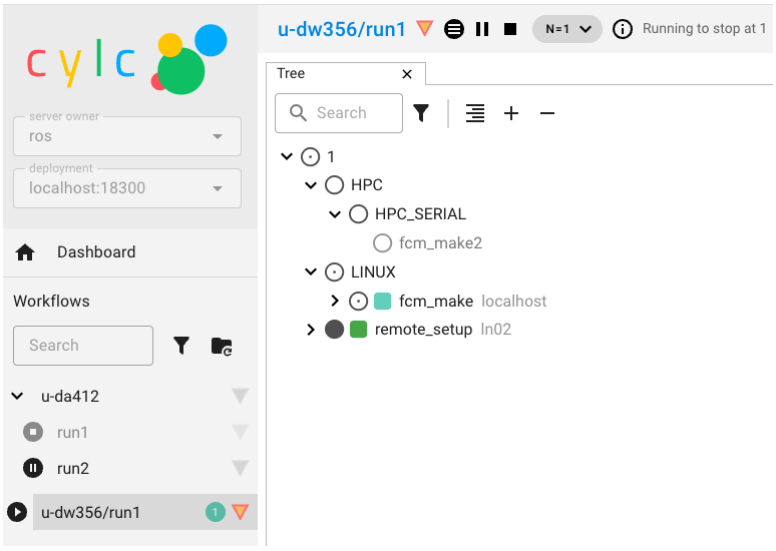
The Cylc GUI is a monitoring and control application that runs in a web browser. The Cylc GUI has different views you can use to examine your workflows, including a menu to allow you to switch between workflows. You only need to have one instance of the Cylc GUI open.

To use the Cylc GUI for workflows on ARCHER2 you need to do some setup first. Please follow the "Setting up the Cylc GUI" instructions in Chapter 1.

To start the Cylc UI, in your local desktop terminal window type `puma-ui` and you should see similar to the following:

```
$ puma-ui
################################################################################
------------------------------Welcome to PUMA2---------------------------------
################################################################################
[C 2026-01-22 08:25:30.367 ServerApp]

To access the server, open this file in a browser:
  file:///home/n02/n02/ros/.cylc/uiserver/info_files/jpserver-1094362-open.html
Or copy and paste one of these URLs:
  http://localhost:20522/cylc?
↪token=700ab2be96800177d03df31b8140857cab02b9632af45a1d
  http://127.0.0.1:20522/cylc?
↪token=700ab2be96800177d03df31b8140857cab02b9632af45a1d
[W 2026-01-22 08:25:44.242 ServerApp] The websocket_ping_timeout (999999) cannot␣
↪be longer than the websocket_ping_interval (290).
  Setting websocket_ping_timeout=290
```

Copy and paste one of the URLs listed into your web browser and you should then see your Cylc GUI load.



Using the Cylc TUI

This is a command line version of the GUI. It can be used to monitor and control any workflows running under your user account, trigger tasks, access log files and perform other common activities.

To start the Cylc TUI on PUMA2 type:

```
puma$ cylc tui <workflow_name>
```

You interact with the TUI by selecting a task name, either by using the up & down arrows or the mouse and then pressing the <Enter> key to bring up a menu of actions.

Please take some time now to familiarise yourself with both the Cylc TUI & Cylc GUI.

4.4 Looking at the queues on ARCHER2

It's likely that whilst you have been looking at the GUIs your workflow will have finished. If this is the case run it again before continuing with this section.

Now, let's log into ARCHER2 and learn how to look at the queues.

Run the following command:

```
squeue -u <archer2-user-name>
```

This will show the status of jobs you are running. You will see output similar to the following:

```
ARCHER2> squeue -u ros
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
            148599  standard u-cc519.      ros  R       0:11      1 nid001001
```

At this stage you will probably only have a job running or waiting to run in the serial queue. Running `squeue` will show all jobs currently on ARCHER2, most of which will be in the parallel queues.

Once your workflow has finished running the Cylc GUI/TUI will go blank and you should get a message in the bottom left hand corner saying `Stopped with succeeded`.

Tip

Cylc is set up so that it *polls* ARCHER2 to check the status of the task, every 5 minutes. This means that there could be a maximum of 5 minutes delay between the task finishing on ARCHER2 and the Cylc GUI/TUI being updated. If you see that the task has finished running but Cylc hasn't updated then you can manually poll the task by selecting it and then selecting *Poll* from the pop-up menu.

4.5 Standard Workflow Output

The output from a standard workflow goes to a variety of places, depending on the type of the file. On ARCHER2 you will find all the output from your run under the directory `~/cylc-run/<workflow-name>`, where `<workflow-name>` is the name of the workflow (e.g. u-dp063). This is actually a symbolic link to the equivalent location in your `/work` directory (E.g. `/work/n02/n02/<username>/cylc-run/<workflow-name>`.

4.5.1 Task output

On PUMA2, navigate to the cylc-run directory for the workflow:

```
cd ~/cylc-run/<workflow-name>
ls
```

By default, cylc will create a new numbered run directory each time you run the workflow:

```
puma2$ cd ~/cylc-run/u-dp063
puma2$ ls
_cylc-install/  run1/  run2/  run3/  runN@
```

The most recent `runX` directory is symlinked to `runN`.

Go to the workflow you have just run and into the log directory:

```
cd runN/log/job/1
ls
```

You will see directories for each of the tasks in the workflow. For this workflow there are 5 tasks: `remote_setup` (ARCHER2 directory setup), `fcm_make` (code extraction), `fcm_make2` (compilation), `recon` & `atmos`. Try looking in one of the task directories:

```
cd recon/NN
ls
```

Here `NN` is a symbolic link created by Rose pointing to the output of the most recently run. You will see several files in this directory. The `job.out` and `job.err` files are the first places you should look for information when tasks fail.

Tip

You can use the command line to view scheduler or job logs without having to find them yourself on the filesystem:

```
cylc cat-log <workflow-name>
cylc cat-log <workflow-name//<cycle-point/<task-name>
```

4.5.2 Compilation output

The output from the compilation is stored on the host upon which the compilation was performed. The output from `fcm_make` is inside the directory containing the build, which is inside the `share` subdirectory.

`~/cylc-run/<workflow-name>/run<X>/share/fcm_make/fcm-make2.log`

If you come across the word “failed”, chances are your model didn’t build correctly and this file is where you’d search for reasons why.

4.5.3 UM standard output

The output from the UM scripts and the output from PE0 of the model are written to the `job.out` and `job.err` files for that task. Take a look at the `job.out` for the `atmos` task, by opening the following file:

`~/cylc-run/<workflow-name>/run<X>/log/job/1/atmos/NN/job.out`

- Did the linear solver for the Helmholtz problem converge in the final timestep?

Job Accounting

The `sacct` command displays accounting data for all jobs that are run on ARCHER2. `sacct` can be used to find out about the resources used by a job. For example; Nodes used, Length of time the job ran for, etc. This information is useful for working out how much resource your runs are using. You should have some idea of the resource requirements for your runs and how that relates to the annual CU budget for your project. Information on resource requirements is also needed when applying for time on the HPC.

Let's take a look at the resources used by your copy of `u-dp063` run.

- Locate the SLURM Job Id for your run. This is a 6 digit number and can be found in the `job.status` file in the cylc task log directory. Look for the line `CYLC_BATCH_SYS_JOB_ID=` and take note of the number after the = sign.

Run the following command:

```
sacct --job=<slurm-job-id> --format="JobID,JobName,Elapsed,Timelimit,NNodes"
```

Where `<slurm-job-id>` is the number you just noted above. You should get output similar to the following:

```
ARCHER2-ex> sacct --job=204175 --format="JobID,JobName,Elapsed,Timelimit,NNodes
↪"
       JobID    JobName    Elapsed  Timelimit   NNodes
------------ ---------- ---------- ---------- --------
204175       u-cc519.a+   00:00:23   00:20:00        1
204175.batch      batch   00:00:23                   1
204175.exte+     extern   00:00:23                   1
204175.0     um-atmos.+   00:00:14                   1
```

The important line is the first line.

- How much walltime did the run consume?
- How much time did you request for the task?
- How many CUs (Accounting Units) did the job cost?

Hint

1 node hour currently = 1 CU. See the ARCHER2 website for information about the CU.

There are many other fields that can be output for a job. For more information see the Man page (`man sacct`). You can see a list of all the fields that can be specified in the `--format` option by running `sacct --helpformat`.

4.5.4 Binary output - work and share

By default the UM will write all output to the directory it was launched from, which will be the task's `work` directory. However, all output paths can be configured in the GUI and in practice most UM tasks will send output to one or both of the workflow's `work` or `share` directories.

`~/cylc-run/<workflow-name>/run<X>/work/1/atmos`

or

`~/cylc-run/<workflow-name>/run<X>/share/data`

For this workflow output is sent to the `work` directory.

Change directory to the work space.

Question

- What files and directories are present?

Model diagnostic output files will appear here, along with a directory called `pe_output`. This contains one file for each processor, for both model and reconfiguration, which contain logging information on how the model behaved.

Open one of these files `<workflow-name>.fort6.peXX` in your favourite editor.

The amount of output created by the workflow and written to this file can be controlled in the suite configuration (*um –> env –> Runtime Controls –> Atmosphere only*). For development work, and to gain familiarity with the system, make sure "Extra diagnostic messages" are output. Switch it on in this suite if it isn't already.

It is well worth taking a little time to look through this file and to recognise some of the key phrases output by the model. You will soon learn what to search for to tell you if the model ran successfully or not. Unfortunately, important information can be dotted about in the file, so just examining the first or last few lines may not be sufficient to find out why the model hasn't behaved as you expected. Try to find answers to the following:

Questions

- How many prognostic fields were read from the start file?
- How many boundary layer levels did you run with?
- What was the range of gridpoints handled by this processor?

Check the file sizes of the different file types. The output directory will contain start dumps, diagnostic output files and possibly a core dump file if the model failed and these usually have very different sizes.

SECTION

FIVE

SOLVING COMMON UM PROBLEMS

Aims

In this section you will learn:

- How to troubleshoot common UM errors
- How to stop, reload and restart a workflow

This section exposes you to more typical UM errors and hints at how to find and fix those errors.

You may encounter other errors, often as a result of mistyping, for which solution hints are not provided.

5.1 Set up N96 GA7.0 AMIP workflow

Find and make a copy of suite `u-dp084`.

Firstly make the essential changes required to run the workflow. That is:

- The account code ('n02-training' if you're on an organised training event)
- Your ARCHER2 user name
- The queue to run in.

Hint

Look in the *suite conf* section. For organised training events you will see that this suite is setup to use reservations listed as Tuesday, Wednesday, Thursday; select the appropriate day. For self-study switch off "Use a reservation on ARCHER2" and select the `short` queue.

5.2 Errors resolved in the code extraction

Save the suite and then *Run* it.

The workflow should fail in the `fcm_make_um` task. This is the task that extracts all the required code from the repository including any branches. The failure will be indicated in the Cylc TUI/GUI with a red square and the state `failed`.

Question

- What is the error?

Examine the `job.err` and `job.out` to find the cause of the problem either on the command line or via the cylc GUI or TUI.

Hint

In the Cylc TUI:
- Click on the `fcm_make_um` task
- Press the **<Enter>** key and select *Log*
- Press the **<Enter>** key to bring up the *"Select File"* menu
- Select the file you wish to view
- Press the **<Enter>** again.

In the Cylc GUI:
- Click on the `fcm_make_um` task
- Select *Log* to view the job logs
- From the *"Select File"* dropdown select the file you wish to view.

The error indicates that the branch cannot be found due to an incorrect branch name. You will need to look at the UM code repository through Trac on MOSRS (https://code.metoffice.gov.uk/trac/um/browser) to determine the correct name.

To fix the error go to panel *fcm_make_um –> env –> Sources* and correct the branch name in `um_sources`.

Save the suite.

Now stop the suite and then re-run it.

On the PUMA2 command line type:

```
puma2$ cylc stop <workflow-name>
puma2$ cylc vip <workflow-name>
```

Note

You can also stop the suite from the Cylc TUI or GUI by selecting *<workflow-name>/run1* and selecting *Stop* from the pop up menu.

The suite will fail in the `fcm_make_um` task again.

Question

- What is the error?

Hint

Again look in the `job.err` file. This kind of error results when changes made in two or more branches affect the same bit of code and which the FCM system cannot understand how to resolve.

Question

- Which file does the problem occur in?

In practice, you would need to edit the code branch to fix the problem with the code conflict. To proceed in this case, navigate to *fcm_make_um –> env –> sources* and remove the branch called `vn13.5_training_merge_error` by clicking on it and then clicking the - sign.

Save the suite.

Last time we stopped the suite and then re-ran it, however, it is possible to reload the suite definition and then re-trigger the failed task without first stopping the running suite. To do this change to the suite directory:

```
puma2$ cd ~/roses/<workflow-name>
```

We then reload the suite definition by running the following Cylc command:

```
puma2$ cylc vr <worflow-name>
```

Enter **y** when asked if you wish to **Continue [y/n]**. Wait for this command to complete before continuing.

Finally in the Cylc TUI or GUI select the failed task and then select *Trigger*.

The `fcm_make_um` task will then submit again.

Question

- Is there an error in `fcm_make_um` this time?

If you look in the `job.err` file now it should be empty and the `job.out` file indicates SUCCESS.

5.3 Errors resolved in the compile and run

Questions

- Has the `fcm_make2_um` (compilation) task completed successfully?
- You should have a failure. Open the `job.err` file - what does it indicate?
- Which routine has an error?
- What is the error?
- What line of the Fortran file does it occur on?

In practice, you would need to fix the error in your branch on PUMA2 and then restart the suite. In this case, navigate to *fcm_make_um –> sources* and remove the branch `vn13.5_training_compile_error`. *Save* the suite, *Stop* the failed run and then *Run* it again.

Tip

This time we chose to shutdown the failed suite rather than do a reload. In this scenario we need to redo the code extraction (`fcm_make_um`) step so doing a reload would be slightly more complex; you would need to *Reload* and then *Trigger* both the `fcm_make_um` and the `fcm_make2_um` tasks. With experience you get to know when it's better to do a *Reload* and when to *Stop* a suite.

Note again that the task submitted successfully.

Questions

- Did the `fcm_make2_um` task succeed this time?
- What about the `install_ainitial` task?
- What is the error?
- Does the start dump exist?
- What is the name of the correct start dump?

Hint

Look in the directory where it thinks the start file should be - is there a candidate in there?

Point your suite to the correct start dump. Fixing this problem isn't quite as easy as it sounds. A search in the Rose edit GUI for the dump file name `co764a.da19880901_00_err` will not locate anything. For this suite it is not possible to fix this issue through the GUI, for some other suites you can edit the initial dump location in the panel *um –> namelist –> Reconfiguration and Ancillary Control –> General technical options*.

Suites can be and are set up differently and there will be times when you need to edit the suite definition files directly.

In your suite directory on PUMA2 (`~/roses/<suite-name>`) use `grep -R` to search for the start dump name `co764a.da19880901_00_err` in the suite files. You should see 2 occurrences listed

```
ros@puma2$ grep -r co764a.da19880901_00_error *
site/meto_cray.cylc:{% set AINITIAL = AINITIAL_DIR + 'N96L85/co764a.da19880901_
↪00_error' %}
site/archer2.cylc:{% set AINITIAL = AINITIAL_DIR + 'N96L85/co764a.da19880901_00_
↪error' %}
```

Edit the dump name in the appropriate `.cylc` file for the HPC we are running on, to point to the correct initial dump file.

Hint

This workflow is set up to run on multiple platforms, make sure you edit the file appropriate to ARCHER2.

Reload the suite definition and then *Trigger* the `install_ainitial` task. The task should succeed this time.

Question

- Has the model run successfully?

This time the model should have failed with an error.

Question

- What is the error message?

Hint

Try searching for `ERROR` - you will soon learn common phrases to help track down problems.

Question

- Which PE Ranks signalled the Abort?

In general it can be useful to note which processors failed and then look at the detailed output for those processors. In this scenario, however, all the processors aborted. We'll now take a look at the individual PE output file. Change to the `pe_output` directory for the atmos_main task. This is under `~/cylc-run/<workflow-name>/runX/work/<cycle>/atmos_main/pe_output`.

Open the file called `<workflow-name>.fort6.pe0`. Sometimes extra information about the error can be found in the individual PE output files.

Question

- At what timestep did the error occur?

The error message indicates that the model has suffered a convergence failure in the routine `EG_BICGSTAB_MIXED_PREC`. This basically means that the model was not able to find a solution to the requested accuracy with the amount of effort specified. In this case the failure results from the value chosen for `gcr_max_iterations`. You could try to find what setting similar models use (with the MOSRS repository you have access to all model setups) or looking at the help within `rose edit` may point you in the right direction. Go to *um –> namelist –> UM Science Settings –> Sections 10 11 12 - Dynamics settings –> Solver* and set it to the suggested value. *Save*, *Reload* and *Re-trigger*.

The model should fail with the same error. So what's gone wrong here? We've changed the value of the number of iterations to a recommended value so why didn't it work? The first thing to check is that the new value has indeed been passed to the model. We do this by checking the variable in the namelists which are written by the Rose system. On ARCHER2 navigate to the work directory for the `atmos_main` task (ie. `~/cylc-run/<workflow-name>/runX/work/<cycle>/atmos_main`). In here you will see several files with uppercase names (e.g. `ATMOSCNTL`, `SHARED`), these contain the Fortran namelists which are read into the model. Have a look inside one of them to see the structure. Now search (use *grep*) in these files for the max number of solver iterations variable `gcr_max_iterations`.

Hint

Search for the string `gcr_max_iterations=`.

Question

- What value does it have?
- Is this what you changed it to in the Rose edit GUI?

So why was the change not picked up? Go back to view the setting in the Rose GUI. By the side of the variable `gcr_max_iterations` there is a little icon of a hand on paper, this indicates that there is an *"optional configuration override"* for this variable.

Optional configuration overrides add to or overwrite the default configuration. They are useful to make it easier to switch between different configurations of the model. For example switching between different resolutions.

Click on the icon and the list of overrides appears. You will see that the variable is set to 1 in the *training* override file and it is this value that is being used in the model. Unfortunately optional configuration override files cannot be changed through the GUI so we will need to edit the Rose file directly. Override files for the `um` app live in the directory `~/roses/<suite-id>/app/um/opt`. Open the file `rose-app-training.conf` and edit the value for `gcr_max_iterations`. *Save*, *Reload* and *Re-trigger* the suite.

Check the `gcr_max_iterations` variable in the namelist file again to confirm that it does now have the correct value. This time the model should run successfully. Check the output to confirm that there are no errors. Check that the model converged at all time steps.

SECTION

SIX

FURTHER EXERCISES (1)

Aims

In this section you will learn how to:

- Change the model output logging behaviour
- Change the processor decomposition and output dump frequency
- Add a new STASH request
- Run the Reconfiguration
- Set up a longer running workflow

6.1 Change the model output logging behaviour

Navigate to *um –> namelist –> Top Level Model Control –> Run Control and Time Settings*.

Set `ltimer` to `True`. Timer diagnostics outputs timing information and can be very useful in diagnosing performance problems.

Save and *Run* the suite.

Check the output from processor 0 in the `fort6.pe000` file.

Questions

- Which routine took the most time?
- How many times was `Atm_Step` called?
- How many time steps did the model run for?
- Which PE was the slowest to run AP2 Boundary Layer? Which was the fastest?

Switch on "IO timing"

Hint

Look in the *IO System Settings* panel.

Re-run the model and search for "Total IO Timings" in the `job.out` file.

6.2 Change the processor decomposition

Navigate to *suite conf –> Domain Decomposition –> Atmosphere*.

Questions

- What is the current processor decomposition?
- Why is this not a good way to run the model?

Hint

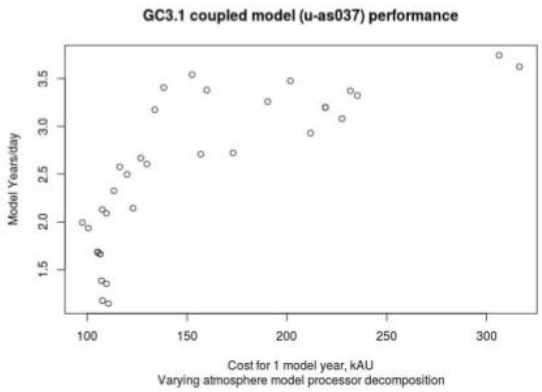
The base ARCHER2 charging unit is a node irrespective of how many cores on the node are being used. ARCHER2 has 128 cores per node, and for the UM each MPI task and OpenMP thread is mapped to a separate core. So in this case we are running 10x16 (x 2 OMP threads) for a total of 320 cores, but are charged for 3 nodes (384 cores).

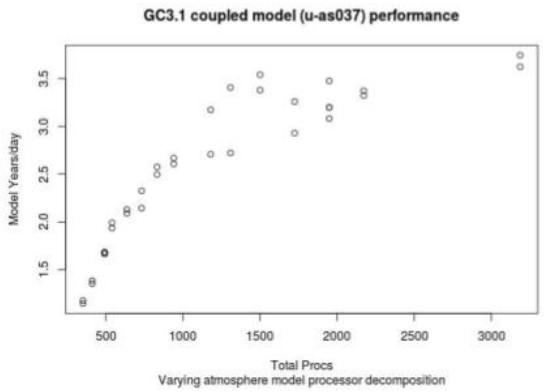
Try experimenting with different processor decompositions (E.g. 10x32, 16x32, etc)

Question

- How do the timings compare to when you ran on 3 nodes?

You can come up with a performance vs processor count curve in this way which might be valuable if you are planning an experiment - it's also worth adding in the CU cost calculation when doing this. An example of this can be seen below:





Note

Running "under populated", i.e. with fewer than the total cores per node, gives access to more memory per parallel task.

Change the processor decomposition to run fully populated on 3 nodes with 2 OpenMP threads.

6.3 STASH

6.3.1 Exploring STASH

Navigate to *um –> namelist –> Model Input and Output –> STASH Requests and Profiles*. Look at the time profiles called `TALLTS` and `T6H`.

Question

- What are they doing?

`TALLTS` says output on every timestep, `T6H` says output 6 hourly.

Look also at some of the other time, domain and usage profiles. The domain profiles determine spatial output and the usage profiles effectively specify a Fortran LUN (Logical Unit Number) on which the associated data is written.

Click on *STASH Requests*. Now change the time profile for all stash output whose `Usage` profile is UPC and `Time` profile is T6H. To do this, click on each diagnostic you wish to change and then click the time profile, a drop-down list should appear containing all the available time profiles. Select `TALLTS`. You can sort the STASH table to make it more convenient to make these changes. Click on the `use_name` column header to sort by usage profile.

6.3.2 STASH validation macro

Several Rose macros have been provided to help verify STASH setup. When you change STASH it is always recommended to run at least the validate macro. The *stash_testmask.STASHTstmskValidate* macro ensures that the STASH output requsted is valid given the science configuration of the app. To put this to the test run the STASH validation macro by selecting *stash_testmask.STASHTstmskValidate* from the list of available macros at the top of the STASH requests panel or alternatively it can be accessed from the *Metadata –> um* menu.

You should see several errors reported - it appears we have asked for diagnostics which are not available. This won't cause the model to fail, however, you could find these diagnostics in the list and switch them off by unchecking the "incl?" column, if you'd like to stop seeing this message.

Save and *Re-run* the suite.

The model should fail with an error message similar to the following:

> **STWORK: Number of fields exceeds reserved headers for unit 15**

This means that the number of output fields exceeds the limit set for a particular stream (the default is 4096 fields); in this case the stream attached to unit 15. To find out what stream unit 15 is take a look in the `fort6.pe000` file and search for "Unit 15". You should see that the file opened on unit 15 is `<suite-id>a.pc19880901`, so this is the `pc` stream. Back in `rose edit` for this suite look at the STASH usage profile for `upc`.

Question

- What is the file ID of the failing output stream?

Now navigate to the window for this stream under *Model Input and Output –> Model Output Streams*. This defines the output stream. You should see confirmation of the base output file name to be `*.pc*`. Changing the reinitialisation frequency by modifying `reinit_step` and/or `reinit_unit` is the best way to fix this header problem. This tells the model to create new output files at a specified frequency, so individual files don't get massively large.

Note

If the model is only exceeding the numer of reserved headers by a small amount it is also possible to just increased the `reserved_headers` size. Overriding the size by a large amount and thus having large numbers of fieldsfile headers can be very inefficient for both runtime and memory. Therefore the recommended way is to change the periodic reinitialisation of the fieldsfiles.

Modify the reinitialisation frequency (you will need to experiment with the numbers) and run the model again. Take a look at the model output files. You should see that you have multiple `*.pc19980901_*` files.

6.3.3 Adding a new STASH request

Let's now try adding a new STASH request to the UM app.

Click the *New* button in the STASH Requests section. A window will appear in order for you to browse all available STASHmaster entries.

By default STASHmaster entries are grouped together by Section code. It is possible to group items by any of the STASHmaster codes using the Group drop down list. The *View* button contains options to display the STASHmaster entry values and/or the column titles with explanation text and to select which columns to show/hide.

Expand the *Gravity wave drag* section. Then change the view by selecting *View –> Show expanded value info*. Try out the other options in the *View* menu to see what effect they have.

Select a STASH item and click *Add* to add it to the list of STASH requests. In the STASH Requests panel click on the empty `dom_name`, `tim_name` and `use_name` fields of the new request and select appropriate profiles from the drop down lists. These lists are populated from the entries of the time, use and domain namelists.

Once you have added a new STASH request, you need to run a macro to generate an index for the namelist. To do so click on the *Macros* button, then select *stash_indices.TidyStashTransform*. A box will pop up listing the changes the editor is going to make, click *Apply*.

Question

- Run the model. Did it work?

6.4 Change the dump frequency

Set the model run length to 1 day.

Hint

Look in the *suite conf –> Run Initialisation and Cycling*.

Note

Hours are represented in the ISO 8601 standard as `PT<num-hours>H` (e.g. `PT1H` represents 1 hour). Days are represented as `P<num-days>D` (e.g. `P10D` represents 10 days)

Reset the STASH output for stream UPC to 6 hourly and the file reinitialisation frequency to 3 hourly.

Navigate to *um –> namelist –> Model Input and Output –> Dumping and Meaning*.

Question

- What is the current dump frequency?

Set the dump frequency to 1 day. *Run* the model.

Question

- How much time was spent in `DUMPCTL`?

Set the dump frequency to 6 hour. *Run* the model.

Question

- What happened to the time spent in `DUMPCTL`?

Important

It is important to understand that writing out model dumps, particularly at higher resolutions, takes up a large amount of time and contributes to the cost. You should think about how frequently you need to output model dumps when setting up your simulations.

6.5 Reconfiguration

Try to find out how to run the reconfiguration only.

Hint

Look in the *suite conf* section.

Try to find out where to request extra diagnostic messages for the reconfiguration output.

Run the reconfiguration only with extra diagnostic messages.

Question

- Look at the `job.out` file. Do you see a land-sea mask?

6.6 Setting up a workflow to cycle

We mentioned in the presentations that the length of an integration will be limited by the time that a model is allowed to run on the HPC (see the ARCHER2 web pages for information about the time limits). Clearly this is no good for much of our work which may need to run on the machine for several months. Cylc and the UM allow for long integrations to be split up into multiple shorter jobs - this is called **cycling**.

Let's run the model for 1 day with 6 hour cycling:

- Set the `Total run length` to 2 days.
- Set the `Cycling frequency` to 1 day.
- Set the `Wallclock time` to 10 minutes.
- Ensure that the model dump frequency is 1 day, in this case.

Save and *Run* the suite.

Note

The cycling frequency must be a multiple of the dump frequency.

The model will submit the first cycle and once that has succeeded you will see the following cycle submitted and run.

Tip

It is always wise, particularly when you plan to run a long integration, that you only run the first cycle initially so that you can check that the model is doing what you expect before committing to a longer simulation. It also enables you to determine how long it takes your model to run and thus be able to calculate an appropriate cycling frequency for your simulation.

6.7 Restarting a suite

Let's now extend this run out to 3 days. Change the `Total run length` to 3 days and *Save* the suite.

Having already run the first day we just want the suite to pick up where it left off and run the remaining day. To do this we *restart* the suite, by typing:

```
puma2$ cylc vr <workflow-name>
```

In either the cylc TUI or cylc GUI you should see the run resuming from where it left off (i.e. from cycle point `19880902T0000Z`).

SECTION

SEVEN

FURTHER EXERCISES (2)

The exercises in this section are all optional. We suggest you pick and choose the exercises that you feel are most relevant to the work you are/will be doing.

Optional Exercises

- Postprocessing (archive and transfer of model data)
- Using IO Servers
- Writing NetCDF output from the UM
- Running the coupled model

Note

Use your copy of suite `u-dp084` for these exercises unless otherwise specified.

7.1 Post-Processing (archive and transfer of model data)

When your model runs it outputs data onto the ARCHER2 `/work` disk (`/projects` on Monsoon3). If you are running a long integration and/or at high resolution data will mount up very quickly and you will need to move the data off of ARCHER2; for example to JASMIN. The post-processing app (postproc) is used within cycling suites to automatically *archive* model data and can be optionally configured to transfer the data from ARCHER2 to the JASMIN data facility. The app archives and deletes model output files, not only for the UM, but also NEMO and CICE in coupled configurations.

Let's try configuring your suite to archive to a staging location on ARCHER2:

- Switch on post-processsing in window *suite conf –> Tasks*

The post-processing is configured under the *postproc* section:

- Select the *Archer* archiving system in window *Post Processing - common settings*.

A couple of new entries will have appeared in the index panel, *Archer Archiving* and *JASMIN Transfer*, identified with the blue dots.

You now need to specify where you want your archived data to be copied to:

- In the *Archer Archiving* panel set `archive_root_dir` to be `/work/n02/n02/<userid>/archive`. The `archive_name` (suite id) will be automatically appended to this.

You will need to run the model for at least 1 day as archiving doesn't work for periods of less than 1 day. Set the `run length` and `cycling frequency` to be 1 day. This should complete in about 5 minutes so set the `wallclock time` to be 10 minutes.

Run the suite.

Once the run has completed go to the archive directory for this cycle (e.g. `/nerc/n02/n02/<userid>/<suiteid>/19880901T0000Z`) and you should see several files have been copied over (e.g `dp084a.pc19880901_00.pp`).

Data files that have been archived and are no longer required by the model for restarting or for calculating means (seasonal, annual, etc) are deleted from the suite `History_Data` directory. Go to the `History_Data` directory for your suite and confirm that this has happened. This run is reinitialising the `pc` data stream every 3 hours and you should see that it has only removed data files for this stream up to 21:00hrs, the `dp084a.pc19880901_21.pp` file is still present. This file contains data for the hours 18-24 and would be required by the model in order to restart. Equally seasonal mean files would not be fully archived until the end of the year, after the annual mean has been created.

Note

The post-processing app can also be configured to transfer the archived data over to JASMIN. Details on how to do this are available on the CMS website: http://cms.ncas.ac.uk/wiki/Docs/PostProcessingApp

7.2 Using IO Servers

Older versions of the UM did not have IO servers, which meant that all reading and writing of fields files went through a single processor (pe0). When the model is producing lots of data and is running on many processors, this method of IO is very inefficient and costly - when pe0 is writing data, all the other processors have to wait around doing nothing but still consuming compute resource (CUs). Later UM versions, including UM 10.5, have IO servers which are processors dedicated to performing IO and which work asynchronously with processors doing the computation.

Here's just a taste of how to get this working in your suite.

Set the suite to run for 1 day with an appropriate cycling frequency, then check that `OpenMP` is switched on as this is needed for the IO servers to work.

Hint

Search for `OMP` in the rose edit GUI

Navigate to *suite conf –> Domain Decomposition –> Atmosphere* and check the number of `OpenMP threads` is set to `2`. Set the number of `IO Server Processes` to `8`.

Save and then *Run* the suite.

You will see lots of IO server log files in `~/cylc-run/<workflow-name>/run1/work/<cycle>/atmos_main` which can be ignored for the most part.

Try repeating the *Change the dump frequency* experiment with the IO servers switched on - you should see much faster performance.

7.3 Writing NetCDF output from the UM

Until UM vn10.9, only fields-file output was available from the UM - bespoke NetCDF output configurations did exist but not on the UM trunk. The suite used in most of these Section 7 exercises is vn11.7, hence supports both fields-file and NetCDF output data formats.

7.3.1 Enable NetCDF

Make sure that `IO Server Processes` variable is set to `0`.

Navigate to *um –> namelist –> Model Input and Output –> NetCDF Output Options* and set `l_netcdf` to `true`. Several fields will appear which allow you to configure various NetCDF options. For this exercise, leave them at their chosen values.

7.3.2 Set NetCDF Output Streams

Expand the *NetCDF Output Streams* section. A single stream - `nc0` - already exists; select it to display its content. As a useful comaprison, expand the *Model Output Streams* section and with the middle mouse button select *pp0*. Observe that the only significant differences between `pp0` and `nc0` are the values of `file_id` and `filename_base`. Data compression options for `nc0` are revealed if `l_compress` is set to `true`. NetCDF deflation is a computationally expensive process best handled asynchronously to computation and as yet not fully implemented through the UM IO Server scheme (but under active development.) For many low- to medium-resolution models and, depending precisely on output profiles, high-resolution models also, use of UM-NetCDF without IO servers still provides significant benefits over fields-file output since using it avoids the need for subsequent file format conversion.

Right-click on *nc0* and select *Clone this section*. Edit the settings of the newly cloned section appropriately to make the new stream similar to `pp1` (ie. edit `filename_base` and all the reinitialisation variables). It is sensible to change the name of the new stream from `1` to something more meaningful, `nc1` for example (right click on `1`, select *Rename a section*, and change . . . nc(1) to . . . nc (nc1)).

7.3.3 Direct output to the nc streams

Expand *STASH Requests and Profiles*, then expand *Usage Profiles*. Assign nc streams to usage profiles - in this suite, UPA and UPB are assigned to `pp0` and `pp1` respectively (where can you see this?). Edit these Usage profiles to refer to `nc0` and `nc1` respectively. Run the STASH Macros (if you need a reminder see Section 6), save the changes, and run the suite. Check that the NetCDF output is what you expected.

Try adding more nc streams to mimic the pp stream behaviour.

7.4 Running the coupled model

The coupled model consists of the UM Atmosphere model coupled to the NEMO ocean and CICE sea ice models. The coupled configuration used for this exercise is the UKESM Historical configuration with an N96 resolution for the atmosphere and a 1 degree ocean - you will see this written N96 ORCA1.

7.4.1 Checkout and run the suite

Checkout and open the suite `u-dw272`. The first difference you should see is in the naming of the apps; there is a separate build app for the um and ocean, called `fcm_make_um` and `fcm_make_ocean` respectively. Similarly there are separate apps for the atmos and ocean model settings, called `um` and `nemo_cice`.

Make the usual changes required to run the suite (i.e. set username, account code, queue). If you are following the tutorial as part of an organised training event, select one of the special queues, otherwise, select to run in the `short` queue.

Check that the suite is set to build the UM, Ocean, and Drivers as well as run the reconfiguration and model.

Run the suite.

7.4.2 Exploring the suite

Whilst the suite is compiling and running which will take around 40 minutes, take some time to look around the suite.

Questions

- How many nodes is the atmosphere running on?
- How many nodes is the ocean running on?
- What is the cycling frequency?

The version of NEMO used in this suite (and most suites you will come across) uses the XML IO Server (XIOS) to wite its diagnostic output. XIOS runs on dedicated nodes (one node in this case). Running `squeue` will show three status entries corresponding to the Atmosphere, Ocean, and XIOS components of the coupled suite. XIOS is running in `multiple-file` mode with 6 servers.

Question

- Can you see where the NEMO model settings appear?

Look under *Run settings (namrun)*. The variables `nn_stock` and `nn_write` control the frequency of output files.

Question

- How often are NEMO restart files written?

Hint

The NEMO timestep length is set as variable `rn_rdt`

Further Reading

NEMO, SI3, CICE (an alternative, widely used sea-ice model) and XIOS are developed separately from the UM, and you should have seen that they work in very different ways. See the following websites for documentation:

- http://oceans11.lanl.gov/trac/CICE
- http://www.nemo-ocean.eu/
- https://zenodo.org/records/7534900
- https://forge.ipsl.jussieu.fr/ioserver

7.4.3 Output files

Log files

NEMO logging information is written to:

```
~/cylc-run/<workflow-name>/run1/work/<cycle>/coupled/ocean.output
```

If the model fails some error messages may also be written to the file `~/cylc-run/<workflow-name>/run1/work/<cycle>/coupled/debug.root.01` or `debug.root.02`

When something goes wrong with the coupled model it can be tricky to work out what has gone wrong. NEMO errors may not appear at the end of the file but will be flagged with the string `E R R O R`.

Restart files

Restart files go to the subdirectory `NEMOhist` in the standard data directory `~/cylc-run/<workflow-name>/run1/share/data/History_Data`.

Diagnostic files

Diagnostic files are left in the `~/cylc-run/<workflow-name>/run1/work/<cycle>/coupled/` directory.

In this example,

NEMO diagnostic files are named `nemo_<workflow-name>o*grid_[TUVW]*`. SI3 diagnostic files are named `si3_<workflow-name>io*icemod*`. To see what files are produced, run:

```
archer2$ ls nemo_dw272o*grid*
archer2$ ls si3_dw272o*icemod*
```

> **Note**
>
> The coupled atmos-ocean model setup is complex so we recommend you find a suite already setup for your needs. If you find you do need to modify a coupled suite setup please contact NCAS-CMS for advice.

7.4.4 Rebuild NEMO

The ocean and ice model components start with single start files (see for example, *si3->Restart files*), but notice in `share/data/History_Data/NEMOhist`, the model has written multiple start files. Type:

```
archer2$ ls  dw272o_19781001_restart_ice*
```

Each processor writes its own start file. This means you can not run on a different ocean/si3 processor decomposition with these files. NEMO provides a utility to combine the individual processor files into a single start file, convenient for moving around and enabling use with differing ocean/ice processor decompositions.

Try running:

```
archer2$ rebuild_nemo dw272o_19781001_restart_ice 108
```

Have a look at the file it created and compare with one of the multiple processor files (using ncdump for example.)

SECTION

EIGHT

ROSE/CYLC EXERCISES

8.1 Differencing suites

Currently there is no Rose tool to difference two suites. Since a suite consists of text files it is simply a matter of making sure all the Rose configuration files are in the common format by running `rose config-dump` on each suite and then running `diff`.

We will difference your copy of the GA9.0 suite with the original one:

```
puma2$ cd ~/roses
puma2$ rosie checkout u-dp084
puma2$ rose config-dump -C u-dp084
puma2$ rose config-dump -C <your-suitename>
puma2$ diff -r u-dp084 <your-suitename>
```

Question

- Are the differences what you expected?

8.2 Graphing a suite

Todo

Find another suite for this task

When developing suites, it can be useful to check what the run graph looks like after jinja evaluation, etc.

The GA9.0 suite that we have been working with is very simple so we shall graph a nesting suite which is more complex. To do this without running the suite:

```
puma2$ rosie checkout u-ce122
puma2$ cd ~/roses/u-ce122
puma2$ rose suite-run -l --name=u-ce122 # install suite in local cylc db only
puma2$ cylc graph u-ce122               # view graph in browser
```

A window containing the graph of the suite should appear. By default tasks in the same family are grouped together. Click the *Ungroup all families* button at the top of the window to expand the graph to view all tasks within this suite.

8.3 Exploring the suite definition files

Change to the `~/roses/<suite-id>` directory for your copy of `u-dp063`.

Open the `flow.cylc` file in your favourite editor.

Look at the `[scheduling]` section. This contains some Jinja2 variables (`BUILD` & `RECON`) which allow the user to select which tasks appear in the dependency graph. The dependency graph tells Cylc the order in which to run tasks. The `fcm_make` and `recon` tasks are only included if the `BUILD` and `RECON` variables are set to true. These variables are located in the `rose-suite.conf` and can be changed using the rose edit GUI or by directly editing the `rose-suite.conf` file. When you run a suite, a processed version of the `flow.cylc` file, with all the Jinja2 code evaluated, is placed in your workflow's `cylc-run` directory.

Question

- Take a look at the `flow-processed.cylc` file for your suite.

Hint

Go to directory `~/cylc-run/<workflow-name>/runX/log/config`.

Change the values of `BUILD` and `RECON` and re-run your suite.

Question

- Look at the new `flow-processed.cylc` file. Can you see how the graph has changed?

Make sure that you leave the suite with `BUILD=false` before continuing.

As we saw earlier when changing the path to the start dump, some settings can't be changed through the rose edit GUI. Instead you have to edit the suite definition files directly.

Question

- Can you find where the atmos processor decomposition is set for this suite?

Change atmos processor decomposition to run on 2 nodes. Run the suite.

Question

- Did it work? If not, what error message did you get?

Hint

Look in the usual `job.out/job.err` or it may be in the `job-activity.log` file.

You will get an error if the processor decomposition of the model does not match the number of tasks and nodes requested by Slurm. For details on the Slurm batch system on ARCHER2 see: https://docs.archer2.ac.uk/user-guide/scheduler/).

In the `[[atmos]] [[[directives]]]` section, set `--nodes=2` and `--ntasks=256` to tell the Slurm scheduler that you require 2 nodes and a total of 256 MPI tasks.

Questions

- The suite should run this time. Did it run on 2 nodes as requested?
- How much walltime has been requested for the reconfiguration?

Now take a look at the `flow.cylc` file for your other suite (the one copied from `u-dp084`). See how it differs. This one is set up to run on multiple platforms.

Questions

- Can you see the more complex dependency graph?
- Why do you not need to adjust the Slurm directives to change the processor decomposition in this suite?
- Can you see where to change the reconfiguration walltime for this suite?

Further Information

This has just given you a very brief look at the suite definition files. More information can be found in the cylc documentation: Writing Workflows

8.4 Removing workflows

If we don't clear up worfklows when we are finished with them disk usage will mount up and you will eventually exceed your quota. As we have seen when running our workflows, files are written to multiple locations; PUMA2, ARCHER2 `/home`, `/work` and potentially other locations. The directory setup includes symlinks which means you can't simple do a `rm -r ~/cylc-run/<workflow-name>` on ARCHER2 for instance as this is just a symlink to `/work`. Fortunately, Cylc provides an easy way to clean up a workflow.

Try cleaning up one of your workflows. We suggest the one you ran in Chapter 4; your copy of u-dp063:

```
puma2$ cylc clean <workflow-name>
```

You will see output similar to the following:

```
ros@puma2$ cylc clean u-dw123
Would clean the following workflows:
  u-dw123/run2
Remove these workflows (y/n): y
INFO - Cleaning u-dw123/run2 on install target: archer2
INFO - [archer2]
  INFO - Removing symlink and its target directory: /home/n02/n02/ros/cylc-run/u-
↪dw123/run2 -> /mnt/lustre/a2fs-work2/work/n02/n02/ros/cylc-run/u-dw123/run2
  INFO - Removing directory: /home/n02/n02/ros/cylc-run/u-dw123
  INFO - Removing directory: /mnt/lustre/a2fs-work2/work/n02/n02/ros/cylc-run/u-
```

(continues on next page)

(continued from previous page)

```
↪dw123
INFO - Removing directory: /home/n02/n02/ros/cylc-run/u-dw123/run2
INFO - Removing directory: /home/n02/n02/ros/cylc-run/u-dw123/_cylc-install
INFO - Removing directory: /home/n02/n02/ros/cylc-run/u-dw123
```

See Also

Cylc User Guide: Removing Workflows

8.5 Adding a new app to a suite

A Rose application or "Rose app" is a Rose configuration for running an executable command, encapsulating details such as scripts, programs and settings.

To add a new app to a suite, we first create a directory to hold the app files. The main details are specified in a configuration file `rose-app.conf`. We may also specify some metadata to tell the general user what inputs to the task mean (this goes under a `meta/` sub-directory or we may reference some standard metadata held elsewhere). Any scripts or executables needed by the new app can be added into an app `bin/` directory. General scripts that aren't specific to the app should go in the *suite* `bin/` directory.

Remember to `fcm add` any new files that you add to the suite so they will be added to the repository when you next commit.

In order to actually run the app, we need to add a new "task" to the suite which involves editing the suite configuration file `suite.rc`. We need to specify 3 things:

1. How the new task relates to other tasks, specifically, which task will trigger it and which task will follow it;
2. What the task will run (i.e which app); and
3. How the task will run (i.e. which computer and the resources it will need).

In this example, we will add an app that prints `Hello World`, which will execute after the reconfiguration and before the main model. We will add the app to your copy of `u-cc654`.

8.5.1 Create the Rose application directory

Make sure the Rose edit GUI for your suite is closed. `cd` into the suite `app/` directory and create a new directory called `new_app`

```
puma2$ cd ~/roses/<SUITEID>/app
puma2$ mkdir new_app
```

8.5.2 Create the Rose app configuration file

Change into the `new_app` directory and create a blank app configuration file called `rose-app.conf`:

```
puma2$ touch rose-app.conf
```

Start the Rose editor (remember you need to be in the top level of the suite directory). You should now see the new application listed in the left hand panel. At this point it is an empty application and is not integrated

into the task chain. Click on *new_app* to load the app and then the little triangle to the left of *new_app* to expand its contents.

> **Tip**
>
> You may need to select *View* > *View Latent Pages* to see the little triangle

Everything is greyed out. Click on *command* to see the command page and then click the + sign next to `command default`. Again you may need to select *View -> View Latent Variables* to see it. Select *add to configuration* to add a command to the application. Enter `echo "Hello World"` in the `command default` box. *Save* this and then have a look at the contents of the `rose-app.conf` file to see the effect.

8.5.3 Add a new task to the suite definition

In order to execute the app, we need to add a new task to the suite workflow. This task executes our new application on a machine that we specify. In this instance we are adding the new task between the reconfiguration and the model run, and the task will be run on ARCHER2 in the serial queue.

To set this up, edit the `suite.rc` file. Under,

```
[scheduling]
   [[dependencies]]
```

find the line

```
{% set INIT_GRAPH = INIT_GRAPH ~ ' => atmos_main' if TASK_RUN else INIT_GRAPH %}
```

and change it to

```
{% set INIT_GRAPH = INIT_GRAPH ~ ' => hello => atmos_main' if TASK_RUN else INIT_
↪GRAPH %}
```

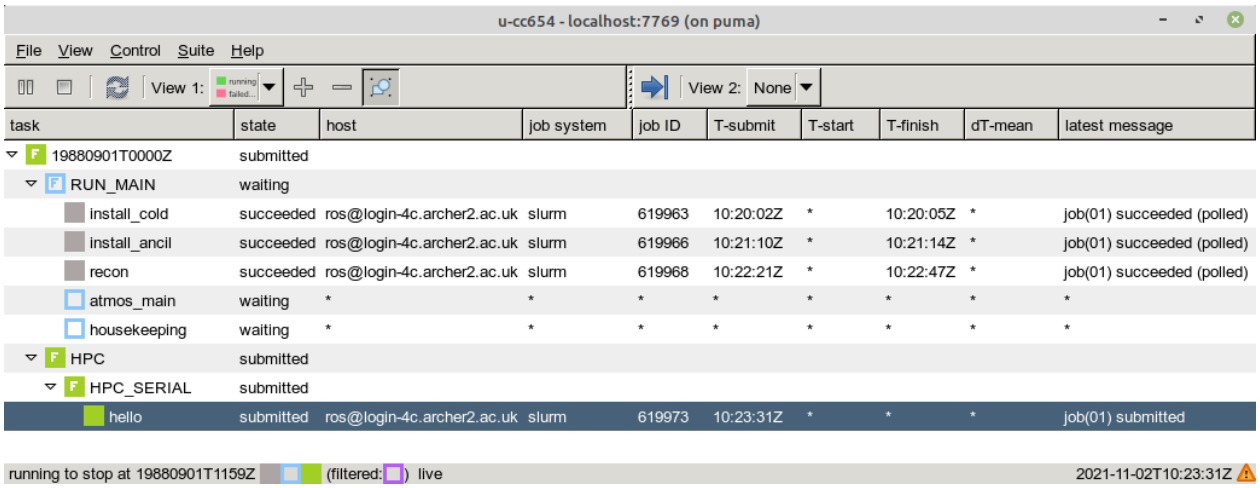
This puts the task `hello` in the right place in the task list.

The next step is to add a definition for the new task. To tell Rose to use one of the apps contained in the suite, we set the environment variable `ROSE_TASK_APP` in the task definition. General task definitions go in the `suite.rc` file and the definitions specific to ARCHER2 in the `site/archer2.rc` file. The queuing system is specific to the host being run on, and there is already a definition for the ARCHER serial queue environment `[[HPC_SERIAL]]` that we can make use of. To run the new application on ARCHER2 in the serial queue and give it two minutes to complete, add the following lines to the `suite.rc` after the definition for `[[recon]]`:

```
[[hello]]
   inherit = HPC_SERIAL
   [[[environment]]]
      ROSE_TASK_APP = new_app
   [[[job]]]
      execution time limit = PT2M
```

8.5.4 Running the new app

We are now ready to go. *Run* the suite. Look at the task graph: `recon` and `atmos_main` are there, but a new hierarchy of tasks has appeared.



Notice that `atmos_main` no longer runs after the reconfiguration, but our new task `hello` does and when that has completed, `atmos_main` starts. The output from the `hello` task can be found in the cylc output directory: `log/job/19880901T0000Z/hello/NN/job.out`.

8.5.5 Extending the app to run a script

A more complex application might involve the execution of a script. To do this we would replace the contents of the `command default` box with the name of the script. Then place the script in the app `bin/` directory.

Now create a `bin/` directory under `new_app/` and `cd` into it. Create a file called `hello.sh` with the contents,

```
#!/bin/bash
echo "Hello, $1!"
```

We will allow the user to select from a variety of planets and say hello. Make it an executable script:

```
chmod +x hello.sh
```

Then we can say `./hello.sh Jupiter` to get it to print "Hello, Jupiter!".

Right click on the greyed out *new_app –> env* in the index panel and click + *Add env*. *Save*, then select *new_app –> env* to view the `env` page, right click on the blank page and select *Add blank variable*. Two boxes appear: enter `PLANET` in the first and `Jupiter` in the second. This adds an environment variable called `PLANET` and sets it to `Jupiter`.

Now change the command from `echo "Hello, World"` to `hello.sh ${PLANET}`.

8.5.6 Testing and Running

The app can be tested in isolation by changing into the `new_app/` directory and executing,

```
rose app-run
```

This should produce output similar to:

```
ros@puma2$ rose app-run
[INFO] export PATH=/home/ros/roses/u-cc654/app/new_app/bin:/home/fcm/rose-2016.
↪11.1/bin:/usr/local/python/bin:
...
[INFO] export PLANET=Jupiter
[INFO] command: hello.sh ${PLANET}
Hello, Jupiter!
```

and also a file `rose-app-run.conf`, which can be deleted.

Now *Run* the suite.

8.5.7 Rose Metadata

Metadata can be used to provide information about settings in Rose configurations. It is used for documenting settings, performing automatic checking and for formatting the rose edit GUI. Metadata can be used to ensure that configurations are valid before they are run.

Metadata for many standard applications, such as `um-atmos`, `fcm_make` are all stored centrally on PUMA2 in `~fcm/rose-meta`. Have a look at this directory.

For our example there are currently no restrictions on the variable `PLANET`. We will now add some metadata to help the user understand what the variable `PLANET` is and what values it is limited to.

Rose provides some tools to quickly guess at the metadata where there is none. Create a directory `meta/` under `new_app/` . Then execute the command,

```
rose metadata-gen
```

This creates a file `rose-meta.conf` in the `meta/` directory. It just says that there is an evironment variable called `PLANET`, but it does not know much about it. Edit this file and add the following lines after `[env=PLANET]`:

```
description=The name of the planet to say hello to.
values=Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune
help=Must be a planet bigger than Pluto - see https://en.wikipedia.org/wiki/
↪Solar_System
```

Now go back to the Rose GUI and select *Metadata* > *Refresh Metadata*. Once the metadata has reloaded, go to the *new_app* –> *env* panel. The entry box for `PLANET` has changed into a drop down list. Pluto is not allowed, presumably because the code cannot handle tiny planets. Right click on the cog next to Planet and select *info* to see the description and allowed values.

8.5.8 References

A fuller discussion of Rose metadata can be found at https://metomi.github.io/rose/2019.01.8/html/tutorial/rose/metadata.html.

Designing a new application may seem a daunting process, but there are numerous existing examples in suites that you can try to understand. For further details, see the Rose documentation at https://metomi.github.io/rose/2019.01.8/html/tutorial/rose/applications.html. There are a collection of built-in applications that you can use for building, testing, archiving and housekeeping - see https://metomi.github.io/rose/2019.01.8/html/api/rose-built-in-applications.html.

SECTION

NINE

POST PROCESSING

This is simply a very basic introduction to some of the more widely used useful tools for viewing, checking, and converting UM input and output data. The tools described below all run on the ARCHER2 login nodes.

9.1 xconv

9.1.1 View data

On ARCHER2 go to the output directory of the global job that you ran previously (the one copied from `u-dp084`). Run `xconv` on the file ending with, for example, `da19880901_04`. This file is an atmosphere start file - this type of file is used to restart the model from the time specified in the file header data.

In the directory above is a file whose name ends in `.astart`; run a second instance of `xconv` on this file. This is the file used by the model to start its run - created by the reconfiguration program in this case.

The `xconv` window lists the fields in the file, the dimensions of those fields (upper left panel), the coordinates of the grid underlying the data, the time(s) of the data (upper right panel), some information about the type of file (lower left panel), and general data about the field (lower right panel.)

Both files have the same fields. Double click on a field to reveal its coordinate data. Check the time for this field (select the "t" checkbox in the upper right panel).

Plot both sets of data - click the *Plot Data* button.

View the data - this shows numerical data values and their coordinates and can be helpful for finding spurious data values.

9.1.2 Convert UM fields data to netCDF

Select a single-level field (one for which `nz=1`), choose *Output format* to be *Netcdf*, enter an "Output file name", and select *Convert*. Information relevant to the file conversion will appear in the lower left panel.

Use `xconv` to view the netcdf file just created.

9.2 uminfo

You can view the header information for the fields in a UM file by using the utility `uminfo` - redirect the output to a file or pipe it to `less`:

```
archer2$ uminfo <one-of-your-fields-files> | less
```

The output from this command is best viewed in conjunction with the Unified Model Documentation Paper F3 which explains in depth the various header fields.

9.3 Mule

Mule consists of a Python API for reading and writing UM files and a set of UM utilities. This section introduces you to some of the most useful UM utilities. Full details of Mule can be found on the MOSRS: https://code.metoffice.gov.uk/doc/um/index.html

Before running the mule commands you will need to load the python environment on ARCHER2 by running:

```
archer2$ module load cray-python
```

9.3.1 mule-pumf

This provides another way of seeing header information, but also gives some information about the fields themselves. Its intended use is to aid in quick inspections of files for diagnostic purposes.

Run `mule-pumf` on the start file - here's a couple of examples on one of Ros' files:

```
archer2$ mule-pumf --print-columns 2 --headers-only \\
                      dp084.astart > ~/mule-pumf-header.out

archer2$ mule-pumf --print-columns 2 dp084.astart > ~/mule-pumf.out
```

- Can you see what the difference is in the output of these 2 commands?

Take a look at the man page (`mule-pumf -h`) and experiment with some of the other options

9.3.2 mule-summary

This utility is used to print out a summary of the lookup headers which describe the fields from a UM file. Its intended use is to aid in quick inspections of files for diagnostic purposes.

Run `mule-summary` on the start file again.

9.3.3 mule-cumf

This utility is used to compare two UM files and report on any differences found in either the headers or field data. Its intended use is to test results from different UM runs against each other to investigate possible changes. Note, differences in header information can arise even when field data is identical. Try out the following:

- Run `mule-cumf` on the two start files referred to above (in the *"View data"* section). You may wish to direct the output to a file.
- Run the same command but with the `--summary` option. This, as the name suggests, prints a much shorter report of the differences.
- Run `mule-cumf` on a file and itself.
- View the help page with `mule-cumf -h` to find view all the available options.

9.4 um-convpp

We have mentioned in the presentations the PP file format - this is a sequential format (a fields file is random access) still much used in the community. PP data is stored as 32-bit, which provides a significant saving of space, but means that a conversion step is required from a fields file (64-bit). The utility to do this is called `um-convpp`. `um-convpp` converts directly from 64-bit files produced by the UM to 32-bit PP files. You must,

however, make sure you are using a version 10.4 or greater - you can check that you are using the right one by typing `which um-convpp`.

Set the stack size limit to unlimited, and add the path to `um-convpp` to your environment - you can also add this to your `~/.profile` so it is available everytime you log in.

```
archer2$ ulimit -s unlimited
archer2$ export PATH=$UMDIR/vn11.2/cce/utilities:$PATH
```

Run `um-convpp` on a fieldsfile (E.g *dp084a.pc19880901_00*)

```
archer2$ cd /home/n02/n02/ros/cylc-run/u-dp084/share/data/History_Data
archer2$ um-convpp dp084a.pc19880901_00 dp084a.pc19880901_00.pp

archer2$ ls -l dp084a.pc19880901*
-rw-r--r-- 1 ros n02 26447872 Nov  2 10:36 dp084a.pc19880901_00
-rw-r--r-- 1 ros n02 20372768 Nov  2 10:47 dp084a.pc19880901_00.pp
-rw-r--r-- 1 ros n02 26476544 Nov  2 10:36 dp084a.pc19880901_06
```

Note the reduction in file size. Now use `xconv` to examine the contents of the PP file.

9.5 cfa

There is an increasing use of python in the community and we have, and continue to develop, python tools to do much of the data processing previously done using IDL or MATLAB and are working to extend that functionality. `cfa` is a python utility which offers a host of features - we'll use it to convert UM fields file or PP data to CF-compliant data in NetCDF format. You first need to set the environment to run `cfa`:

```
archer2$ export PATH=/home/n02/n02/dch/cf-analysis/bin:$PATH
archer2$ cfa -i -o dp084a.pc19880901_00.nc dp084a.pc19880901_00.pp
```

Try viewing the NetCDF file with xconv.

`cfa` can also view CF fields. It can be run on PP or NetCDF files, to provide a text representation of the CF fields contained in the input files. Try it on a PP file and its NetCDF equivalent, e.g.

```
archer2$ cfa -vm dp084a.pc19880901_00.pp | less
Field: long_name:HEAVYSIDE FN ON P LEV/UV GRID (ncvar%UM_m01s30i301_vn1100)
------------------------------------------------------------------------------
Data           : long_name:HEAVYSIDE FN ON P LEV/UV GRID(time(5), air_
↪pressure(17), latitude(145), longitude(192))
Cell methods   : time: point
Axes           : time(5) = [1988-09-01T00:00:00Z, ..., 1988-09-01T03:59:59Z] 360_
↪day
               : air_pressure(17) = [1000.0, ..., 10.0] hPa
               : latitude(145) = [-90.0, ..., 90.0] degrees_north
               : longitude(192) = [0.0, ..., 358.125] degrees_east

Field: long_name:VORTICITY 850 (ncvar%UM_m01s30i455_vn1100)
------------------------------------------------------------
Data           : long_name:VORTICITY 850(time(5), latitude(145), longitude(192))
Cell methods   : time: point
```

(continues on next page)

(continued from previous page)

```
Axes           : air_pressure(1) = [-1.0] hPa
               : time(5) = [1988-09-01T00:00:00Z, ..., 1988-09-01T03:59:59Z] 360_
↪day
               : latitude(145) = [-90.0, ..., 90.0] degrees_north
               : longitude(192) = [0.0, ..., 358.125] degrees_east
```

9.6 CF-python CF-plot

Many tools exist for analysing data from NWP and climate models and there are many contributing factors for the proliferation of these analysis utilities, for example, the disparity of data formats used by the authors of the models, and/or the availability of the underlying software. There is a strong push towards developing and using python as the underlying language and CF-netCDF as the data format. CMS is home to tools in the CF-netCDF stable - here's an example of the use of these tools to perform some quite complex data manipulations. The user is insulated from virtually all of the details of the methods allowing them to concentrate on scientific analysis rather than programming intricacies.

- Set up the environment and start python.

```
archer2$ export PATH=/home/n02/n02/dch/cf-analysis/bin:$PATH
archer2$ python
>>>
```

We'll be looking at some AIRS satellite-retrieved temperature data over sea.

- Import the cf-python library

```
>>> import cf
```

- Read in the AIRS data files

```
>>> f = cf.read('~dch/UM_Training/ta_mon_AIRS-1-0_BE_gn_*.nc')[0]
```

- Inspect the file contents with different amounts of detail

```
>>> f
>>> print(f)
>>> f.dump()
```

Note that the ten AIRS files (one for each year) are automatically aggregated into one field.

- Read in another field produced by a GCM, this has a different latitude/longitude grid to the AIRS data

```
>>> g = cf.read('~dch/UM_Training/tas_Amon_HadGEM3-GC3-1_hist-1p0_r3i1p1f2_
↪gn_185001-201412.nc')[0]
>>> print(g)
```

- Regrid the observed AIRS data (`f`) to the grid of the model field (`g`) over Europe. Use the "conservative" regridding method that preserves area integrals

```
>>> f = f.regrids(g.subspace(X=cf.wi(-10, 40), Y=cf.wi(35, 70)), method=
↪'conservative')
>>> print(f)
```

Note that the latitude and longitude dimensions are now shorter in length.

- Average the regridded field over June-August for each year

```
>>> f = f.collapse('T: mean within years T: mean over years', within_
↪years=cf.jja())
>>> print(f)
```

Note that the time axis is now of length 1.

- Import the cfplot visualisation library

```
>>> import cfplot
```

- Make a default contour plot of the regridded observed data, `f` at the 1000 hPa level

```
>>> f = f.subspace(Z=cf.eq(1000, 'hPa'))
>>> cfplot.con(f)
```

- Make a "blockfill" plot of the regridded observed data, `f`

```
>>> cfplot.con(f, blockfill=True)
```

- Make a default contour plot of the model data, `g` for its first time:

```
>>> g = g.subspace(T=[0])
>>> cfplot.con(g)
```

- Make a "blockfill" plot of the model data, `g`, restricting the view to over Europe

```
>>> cfplot.mapset(lonmin=-10, lonmax=40, latmin=25, latmax=70)
>>> cfplot.con(g, blockfill=True)
```

- Write out the regridded data to disk

```
>>> cf.write(f, 'obs_temperature_Europe_JJA_2003-2010.regridded.nc')
```

This has just given you a taster of CF-Python & CF-Plot, if you would like to try out some more exercises please take a look at https://github.com/NCAS-CMS/cf-tools-training.

SECTION

TEN

APPENDIX A: USEFUL INFORMATION

10.1 UM output

ARCHER2 job output directory:

The standard output and error files (job.out & job.err) for the compile, reconfiguration and run are written to the directory:

```
~/cylc-run/<workflow-name>/run1/log/job/<cycle>/<app>
```

ARCHER2 model output:

By default the UM will write all output (e.g. processor output and data files) to the directory it was launched from, which will be the task's **work** directory. However, all output paths can be configured in the GUI and in practice most UM tasks will send output to one or both of the suite's **work** or **share** directories:

```
~/cylc-run/<workflow-name>/run1/work/1/atmos
~/cylc-run/<workflow-name>/run1/share/data
```

10.2 ARCHER2 architecture

ARCHER2 has two kinds of processor which we commonly use - they have several names, but roughly speaking they are the service processors (several nodes worth) sometimes referred to as the front end, and the compute processors (many many nodes worth) sometimes referred to as the back end. We login to the front end and build the model on the front end. We run the model on the back end. You wouldn't generally have an interactive session on the back end and will submit jobs there through the batch scheduler (slurm).

The UM infrastructure recognises this architecture and will run tasks in the appropriate place.

If you are doing any post-processing or analysis you may wish to submit your own parallel or serial jobs. Intensive interactive tasks should be run on the post-processor nodes (note. these will be available when the full system is in service.)

Consult the ARCHER2 documentation for details (See www.archer2.ac.uk).

10.3 ARCHER2 file systems

ARCHER2, in common with some other HPC systems, such as MONSooN, has (at least) two file systems which have different properties, different uses, different associated policies and different names. On ARCHER2 there are `/home` and `/work`. The `/home` file system is backed up regularly (only for disaster recovery), has relatively small volume, can efficiently handle many small files, and is where we recommend

the UM code is saved and built. The `/home` system can not be accessed by jobs running on the compute processors.

The `/work` file system is optimized for fast parallel IO - it doesn't handle small files very efficiently. It is where your model will write to and read from.

10.4 ARCHER2 node reservations

In normal practice you will submit your jobs to the parallel queue on ARCHER2; the job scheduler will then manage your job request along with all those from the thousands of other users. For this training course, we will be using processor Reservations, whereby we have exclusive access to a prearranged amount of ARCHER2 resource meaning that you will not need to wait in the general ARCHER2 queues. Reservations are specified by a reservation code - e.g. n02-training_266. As an ARCHER2 user you can make a reservation so that you have access to the machine at a time of your choosing - reservations incur a cost overhead (50%), so best used when you are sure you need them.

10.5 Cylc Cheat Sheet

Summary sheet covering most of the major Cylc commands for interacting with a workflow: https://cylc.github.io/cylc-doc/stable/html/user-guide/cheat-sheet.html

SECTION

ELEVEN

APPENDIX B: SSH FAQS

This Sections provides instructions for some common ssh tasks. If you have any problems, contact a member of the CMS team.

11.1 Using an existing ssh agent

If you already have an ssh-agent set up on PUMA2, you can use this one to connect to your ARCHER2 training account. Conversely, after the course you may wish to use the keys you set up for own Archer account.

You can copy your ssh key over to Archer using the `ssh-copy-id` script.

First you need to find the name of the public key in your `.ssh` directory.

```
puma2$ cd ~/.ssh
puma2$ ls
environment.puma2.archer2.ac.uk  id_rsa  id_rsa.pub   known_hosts  ssh-setup
```

The public key ends with `.pub` and will usually be called `id_rsa.pub` or `id_dsa.pub`.

Now run the script to copy the key to your ARCHER2 account, making sure to use the correct name for your key:

```
puma2$ ssh-copy-id -i ~/.ssh/id_rsa.pub <archer-username>@login.archer2.ac.uk
```

You will be prompted for your Archer password.

If successful, you should now be able to login to Archer without a password. If you are prompted for a passphrase you need to re-start your agent - see below.

11.2 Restarting your ssh agent

Normally your ssh agent persists even when you log out of puma2. However, from time to time it can vanish.

If you are prompted for your passphrase, this means the ssh agent has stopped for some reason. The agent *should* have been re-initialised when you logged into PUMA2, but you will need to re-associate your ssh keys to the agent.

To do so, run:

```
puma2$ ssh-add
```

If successful this will prompt for your passphrase:

```
Enter passphrase for /home/<puma2-username>/.ssh/id_rsa_archerum:
```

Sometimes this step will fail with the following error:

```
Could not open a connection to your authentication agent.
```

In this case, the agent is not running. Usually this is beacuse of an environment file. Delete the following:

```
puma2$ rm ~/.ssh/environment.puma2.archer2.ac.uk
```

Then log out of puma2 and back in again. You should hopefully see a message similar to:

```
Initialising new SSH agent...
```

And you should now be able to run ssh-add successfully.

11.3 Regenerating your ssh keys

If you have forgotten your passphrase you will need to regenerate your ssh keys. Before doing so, you will need to tidy up the old keys otherwise the ssh agent can get itself confused.

Go to your `.ssh` directory, and look at the files:

```
puma2$ cd ~/.ssh
puma2$ ls
environment.puma2.archer2.ac.uk  id_rsa  id_rsa.pub   known_hosts  ssh-setup
```

Delete the public and private keys. These will normally be named `id_rsa` and `id_rsa.pub`, or `id_dsa` and `id_dsa.pub`.

You should also delete the `environment.puma2.archer2.ac.uk` file:

```
puma2$ rm id_rsa id_rsa.pub environment.puma2.archer2.ac.uk
```

Next check if you have an agent running:

```
puma2$ ps -flu <puma2-username> | grep ssh-agent
```

If you have an agent running, one or more lines like the following will be returned:

```
15658 ?        00:00:00 ssh-agent
```

The number in the first column is the process-id, pass this to the `kill` command to stop the process, for example:

```
puma2$ kill -9 15658
```

You can now start again, following the *ssh set up instructions*.